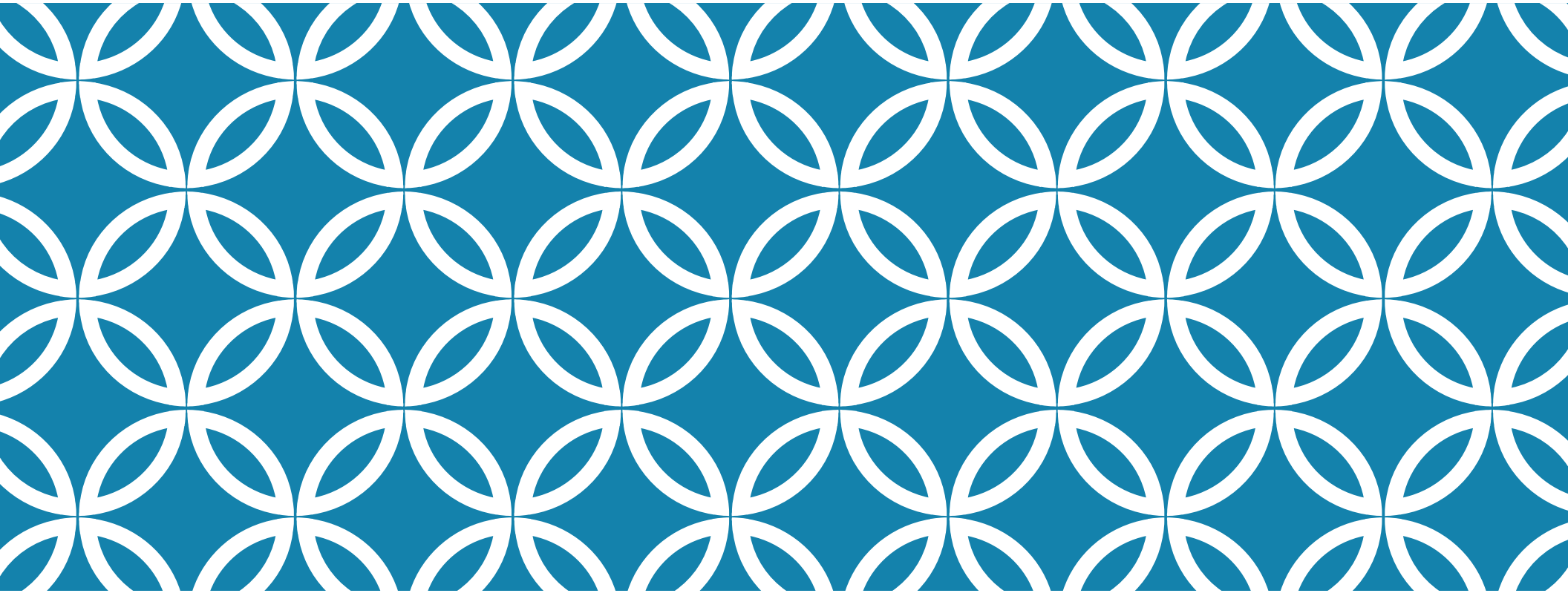




ARTIFICIAL INTELLIGENCE

Lec 06

WHAT WE STUDIED BEFORE!

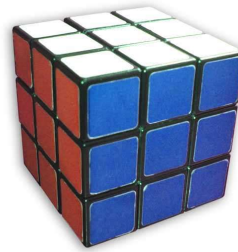
Explore whole state space graph **systematically**.

This systematicity is achieved by

- keeping one or more paths in memory and
- by recording which alternatives have been explored at each point along the path.

When a goal is found, the *path* to that goal also constitutes a *solution* to the problem

LOCAL SEARCH



In many problems, however, the path to the goal is irrelevant. e.g., Rubik's cube.

For such cases, we do a local search e.g., instead of memorizing nodes, we only expand the current node, find the best child, expand it, then find its best child, then expand it and so on.

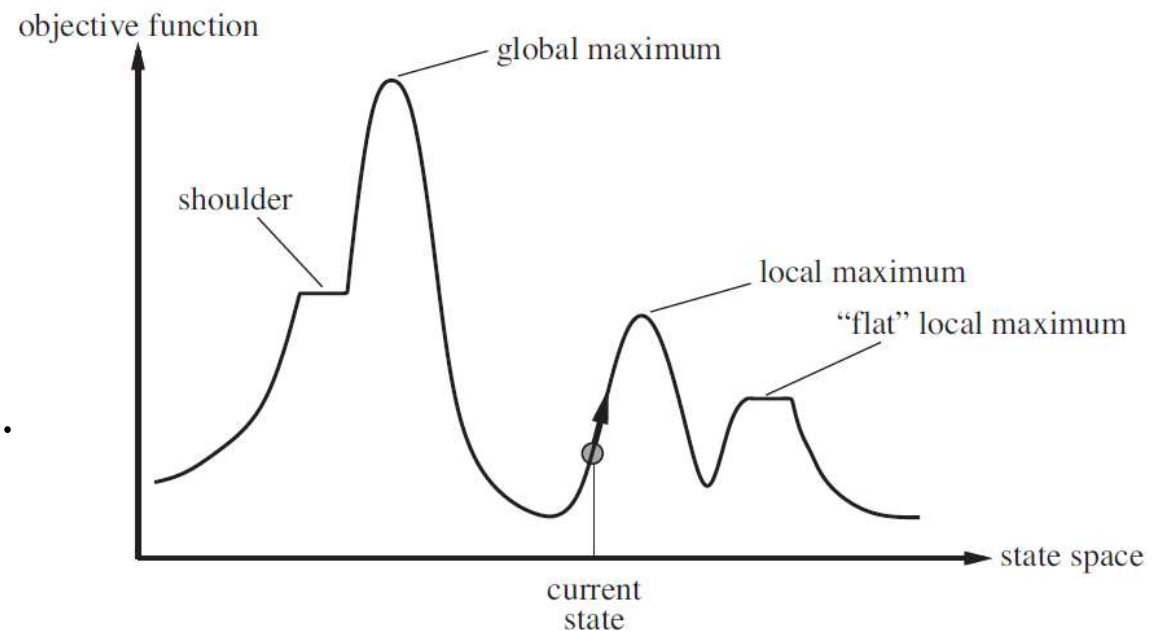
Advantages: low memory and working in a large state space graph (large value of m and d)

STATE SPACE LANDSCAPE

A landscape has

“location” (defined by the state) and

“elevation” (defined by the value of the heuristic cost function or objective function).



OPTIMIZATION

In optimization problems, aim is to find the best state according to an **objective function**

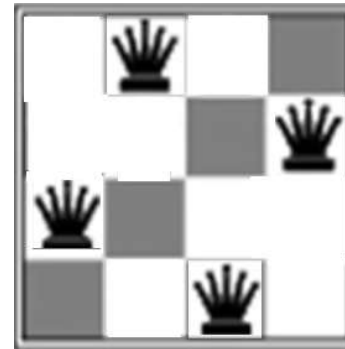
Example: n-queens problem: Put n queens on a $n \times n$ board with no two queens on the same row, column or diagonal.



Initial state (not given,
randomly generated)



Solution 1



Solution 2

HILL CLIMBING / GREEDY LOCAL SEARCH

Objective Function: $h(n)$

$$\sum_{i=1}^4 \text{number of queens in same row, column or diagonal as that of } i^{\text{th}} \text{ queen}$$

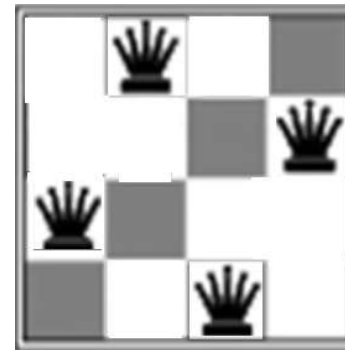
$$2 + 3 + 3 + 2$$

$$= 10$$



Initial state (n)

Find the state n for which this value is minimum
(in this case minimum is 0)

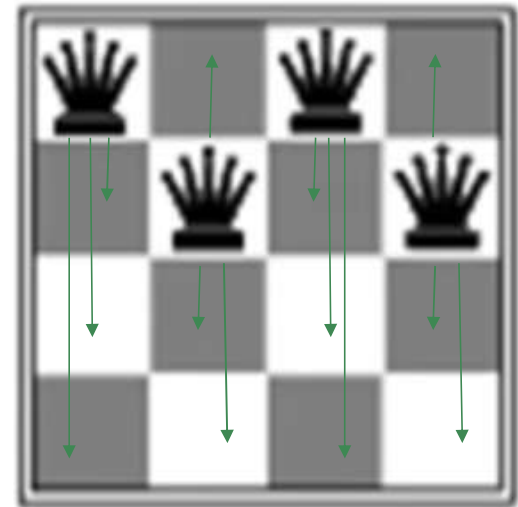


POSSIBLE SUCCESSORS / NEIGHBORS

You can move a queen anywhere within that column

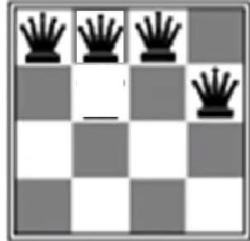
Therefore each state has $4 * 3 = 12$ successor states.

Among these 12 successor states, find the one with lowest value of h and set that successor as the current state; continue until $h=0$!

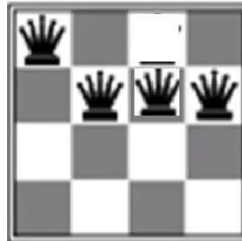




$$h = 2+3+2+3=10$$



$$h = 2+2+3+1=8$$



$$h = 1+3+2+2=8$$



$$h = 3+2+3+2=10$$



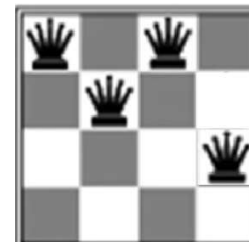
$$h = 2+3+3+2=10$$



$$h = 1+0+2+1=4$$



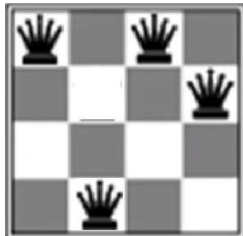
$$h = 2+3+3+2=10$$



$$h = 2+2+2+0=6$$



$$h = 0+2+2+2=6$$



$$h = 1+1+2+1=5$$



$$h = 1+2+0+1=4$$



$$h = 3+3+2+2=10$$

HILL CLIMBING - PSEUDOCODE

function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

loop do

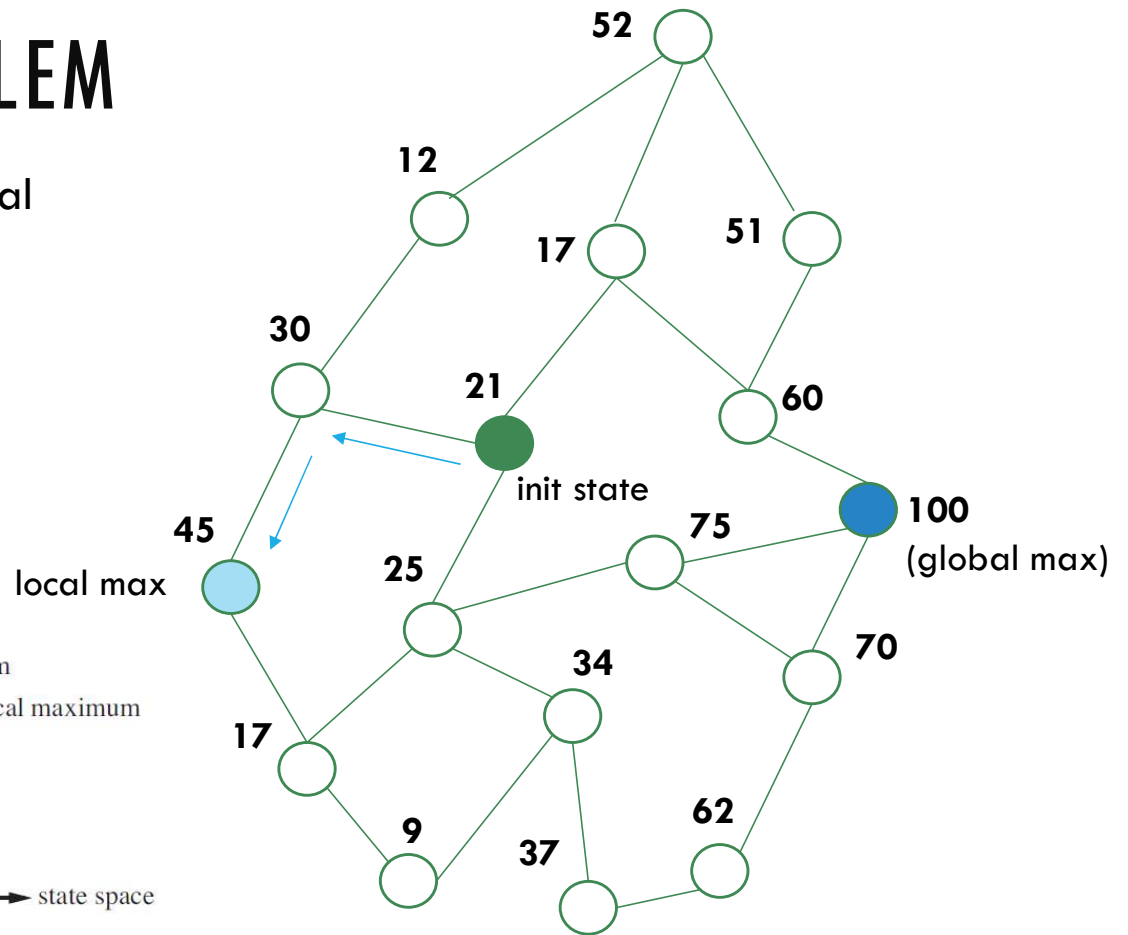
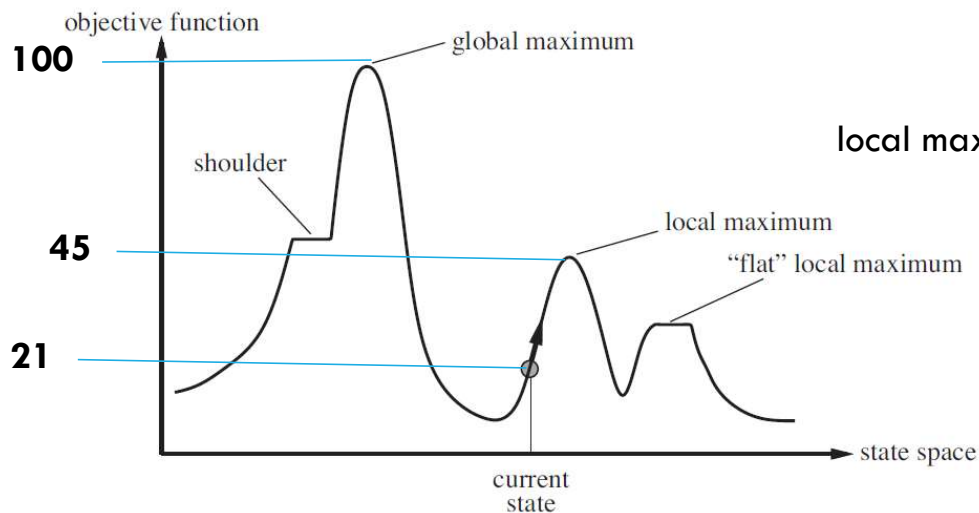
neighbor $\leftarrow$ a highest-valued successor of *current*

if *neighbor*.VALUE $\leq$ *current*.VALUE **then return** *current*.STATE

current $\leftarrow$ *neighbor*

HILL CLIMBING - PROBLEM

Normal hill climbing can get stuck at local maximum/minimum depending upon start/initial/current state.



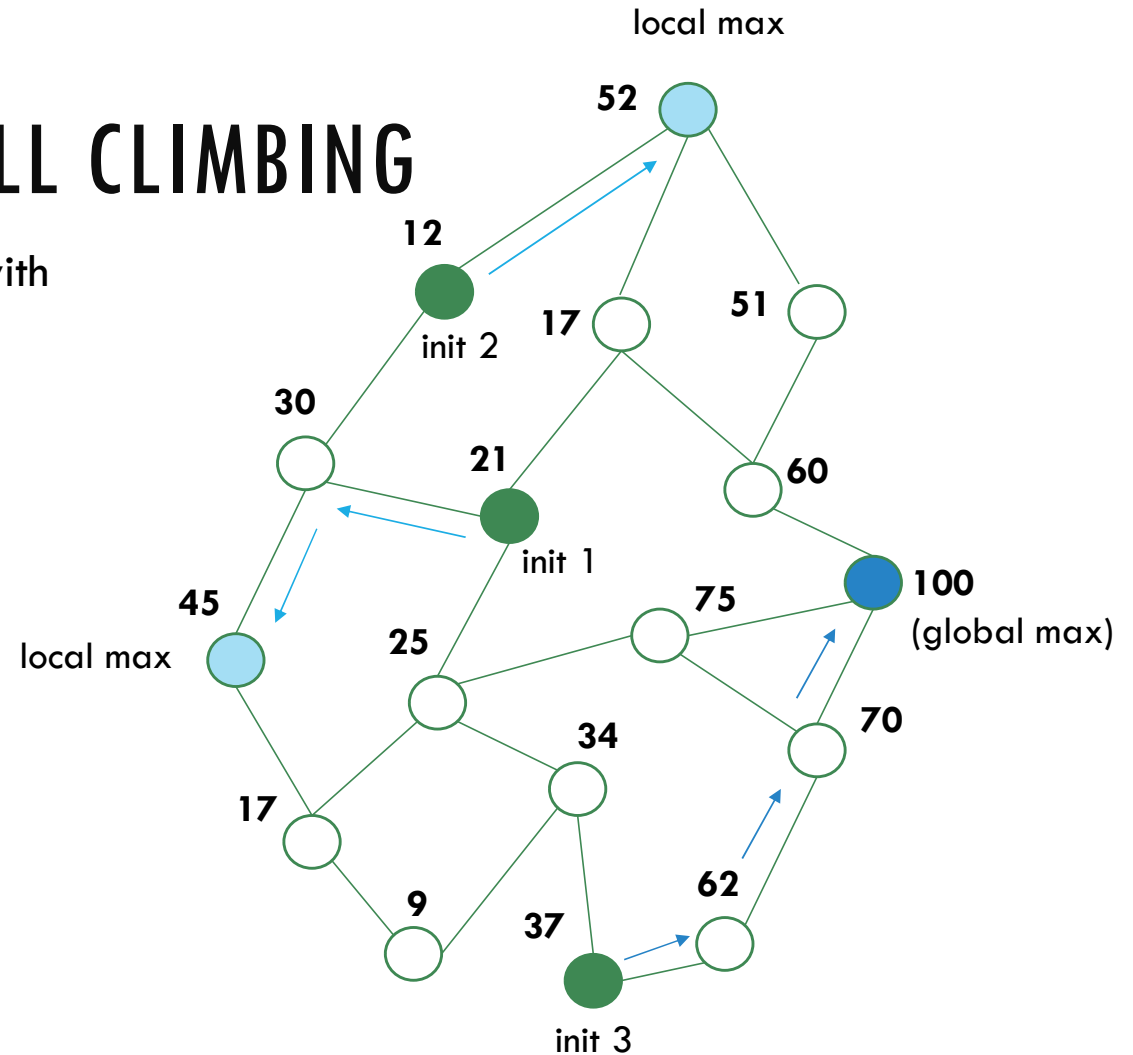
HILL CLIMBING - PROBLEM

Solution(s)

- Random Restart Hill Climbing
- Local Beam Search
- Genetic Algorithms
- Simulated Annealing

RANDOM RESTART HILL CLIMBING

In case of failure in 1st search, restart with a different initial state. Keep repeating the procedure until $h == 100$ state is reached.



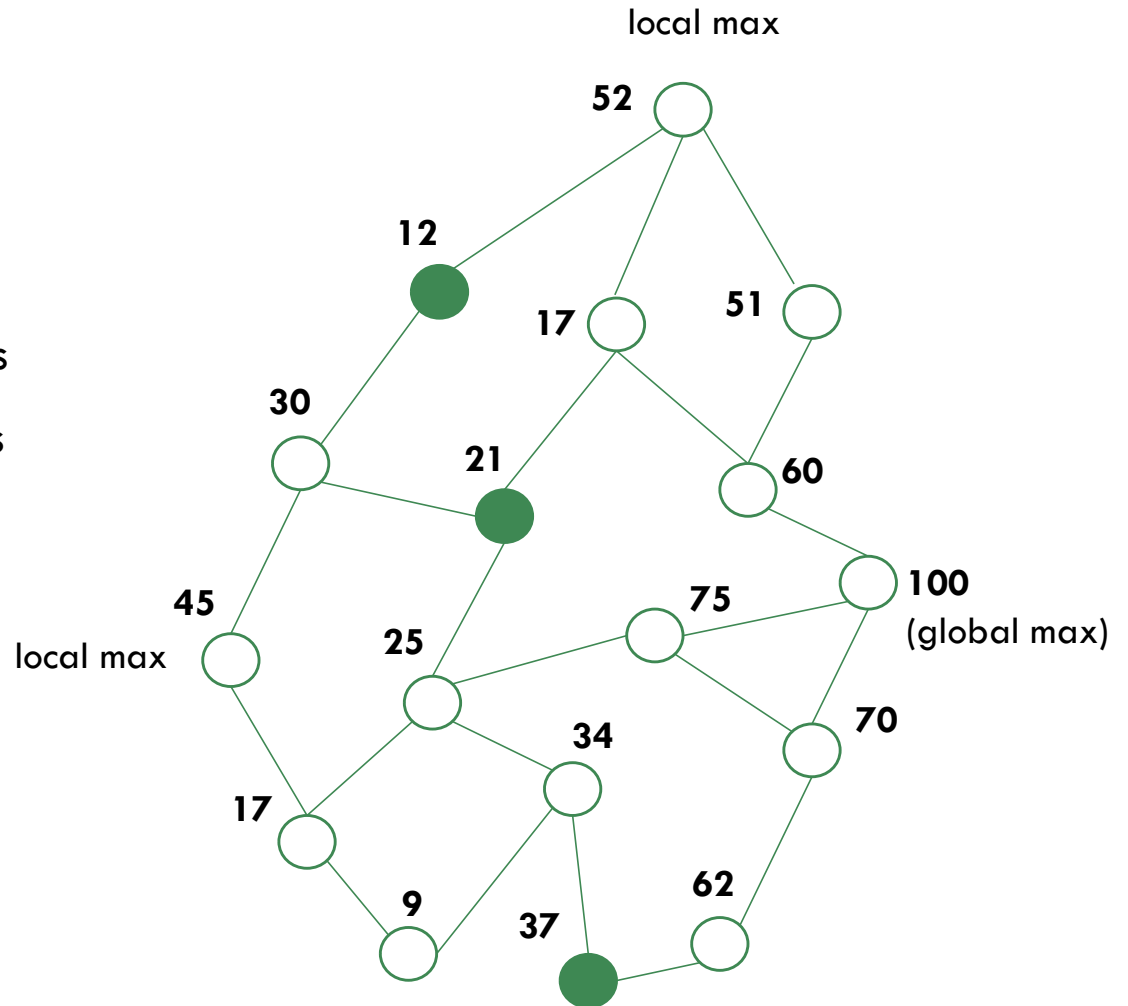
LOCAL BEAM SEARCH

Start with k randomly generated states

Generate all successors of those k states

If no goal found, select best k successors and repeat.

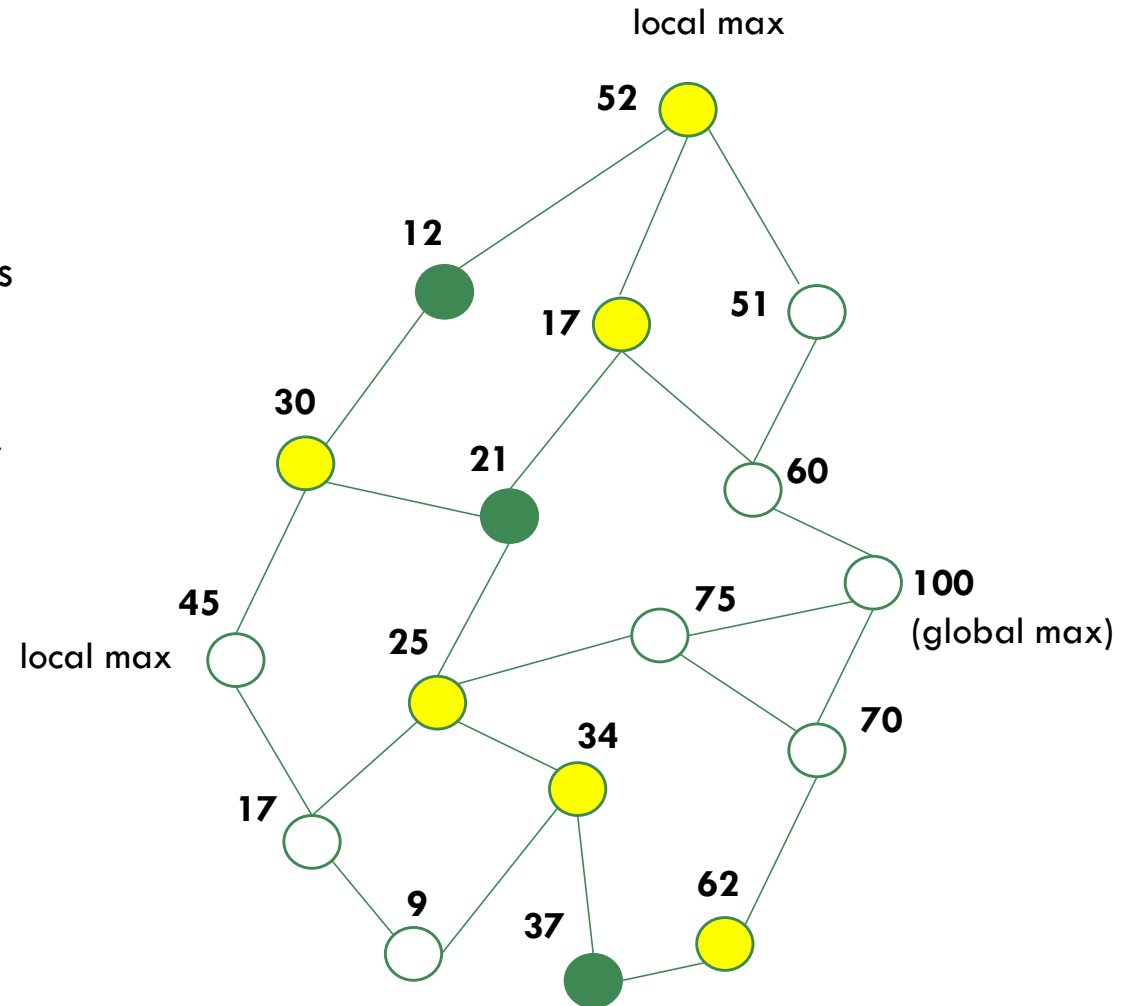
For the tree on the right, $k = 3$



LOCAL BEAM SEARCH

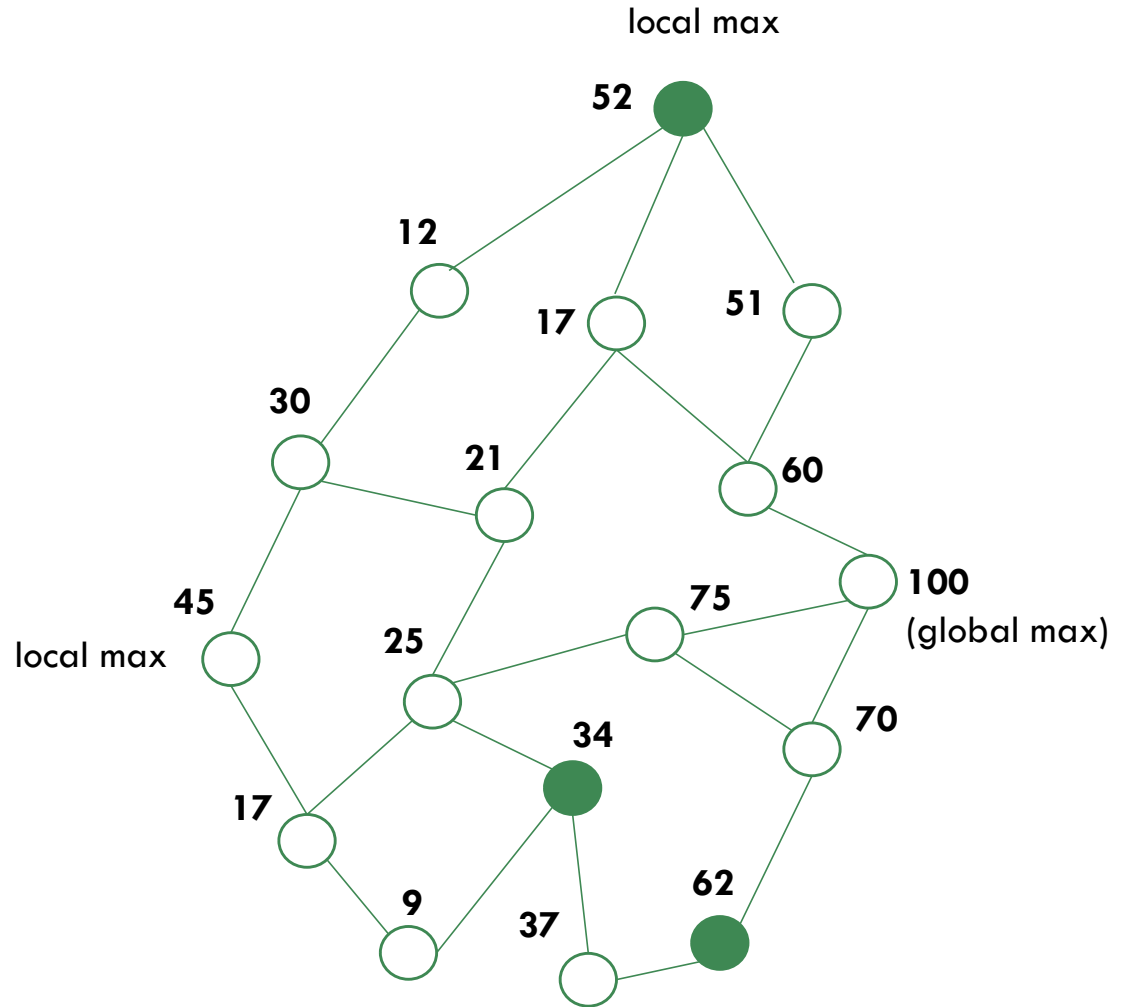
Generate all successors of those k states

Among these successors (yellow) choose, top 3 since $k=3$.



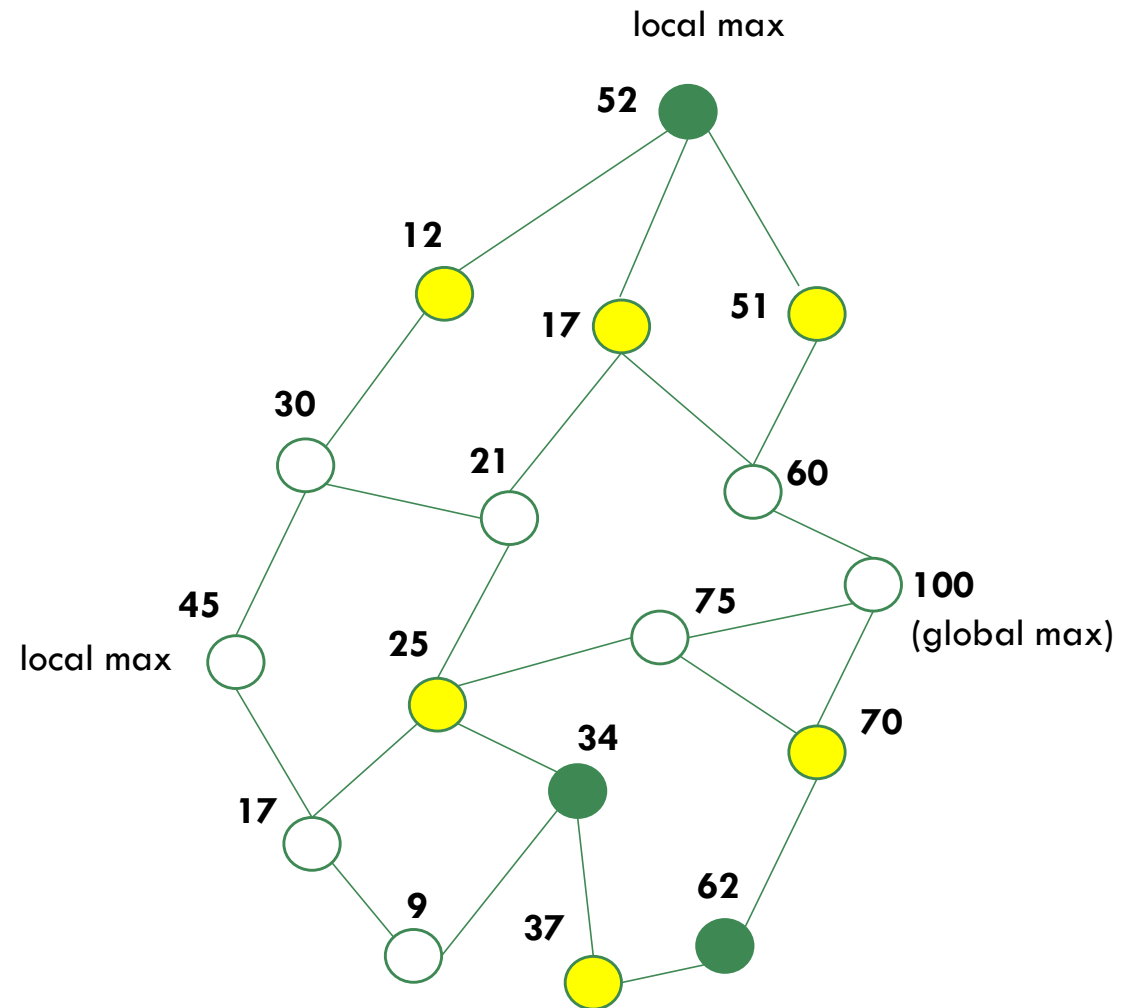
LOCAL BEAM SEARCH

For these 3, repeat the same procedure until state with $h=100$ is reached.



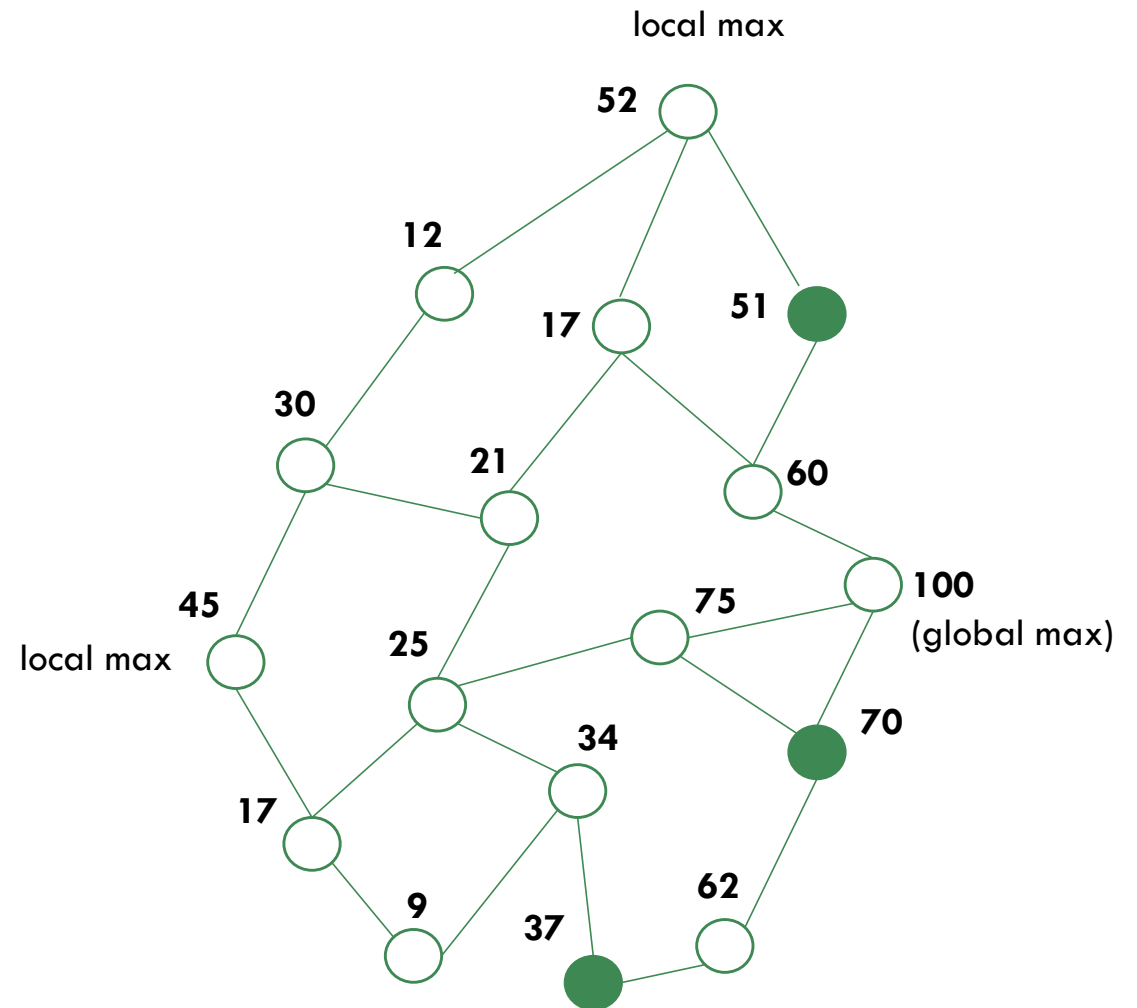
LOCAL BEAM SEARCH

New children are generated



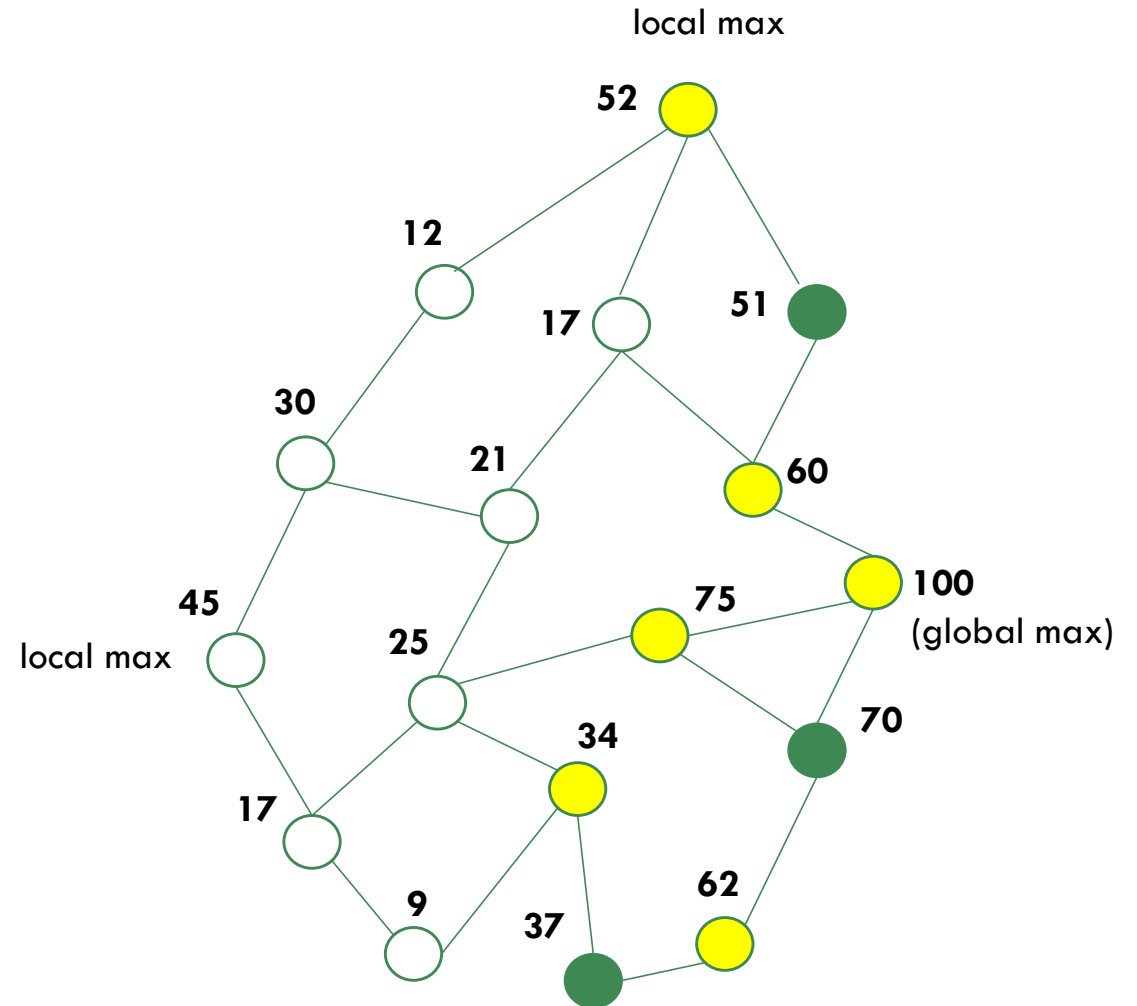
LOCAL BEAM SEARCH

Best 3 are selected.



LOCAL BEAM SEARCH

One of the successors is a goal (global max) !



Simulated Annealing

- Idea: escape local maxima by allowing some “bad” moves
- **But gradually decrease their size and frequency**

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  inputs: problem, a problem
           schedule, a mapping from time to “temperature”
  local variables: current, a node
                    next, a node
                    T, a “temperature” controlling prob. of downward steps

  current ← MAKE-NODE(INITIAL-STATE[problem])
  for t ← 1 to ∞ do
    T ← schedule[t]
    if T = 0 then return current
    next ← a randomly selected successor of current
     $\Delta E \leftarrow \text{VALUE}[\textit{next}] - \text{VALUE}[\textit{current}]$ 
    if  $\Delta E > 0$  then current ← next
    else current ← next only with probability  $e^{\Delta E/T}$ 
```

GENETIC ALGORITHMS

Search technique based upon **natural genetics and natural selection**

Finds the optimal solution i.e., global maximum

Based upon survival of the fittest

“Genetic Algorithms are good at taking large, potentially huge search spaces and navigating them, looking for optimal combinations of things, solutions you might not otherwise find in a lifetime.”

- Salvatore Mangano
Computer Design, May 1995

Genetic Algorithms



FIRST — A BIOLOGY LESSON

A *gene* is a unit of heredity in a living organism

Genes are connected together into long strings called *chromosomes*

A gene represents a specific *trait* of the organism, like eye colour or hair colour, and has several different settings.

- For example, the settings for a hair colour gene may be blonde, black or brown etc.

These genes and their settings are usually referred to as an organism's *genotype*.

The physical expression of the genotype — the organism itself - is called the *phenotype*.

FIRST — A BIOLOGY LESSON

Offsprings inherit traits from parents

An offspring may end up having half the genes from one parent and half from the other
- *recombination*

Very occasionally a gene may be *mutated* — Expressed in an organism as a completely new trait

- For example: A child may have green eyes while none of the parents had

GENETIC ALGORITHM IS

... Computer algorithm

That resides on principles of genetics and evolution



GENETIC ALGORITHMS

Search algorithms based on the mechanics of biological evolution

Developed by John Holland, University of Michigan (1970's)

Provide efficient, effective techniques for optimization and machine learning applications

Widely-used today in business, scientific and engineering circles

GENETIC ALGORITHM

Genetic algorithm (GA) introduces the principle of evolution and genetics into search among possible solutions to given problem

The idea is to simulate the process in natural systems

This is done by the creation within a machine a population of individuals

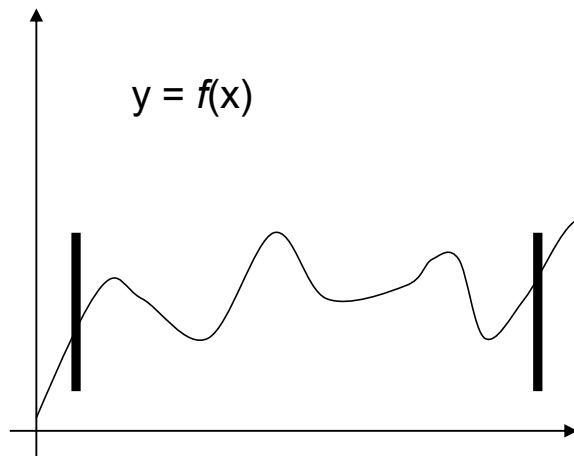
GENETIC ALGORITHM

○ Survival of the fittest

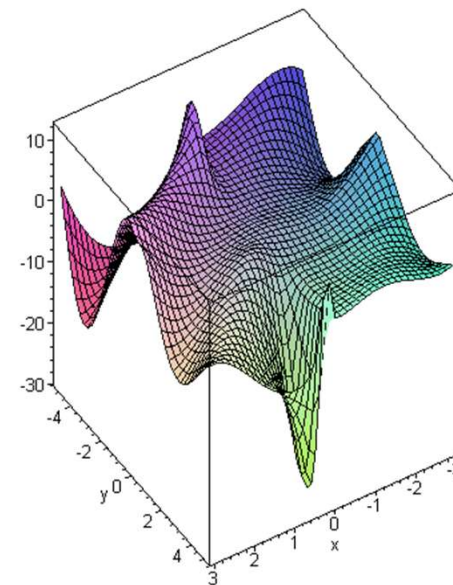
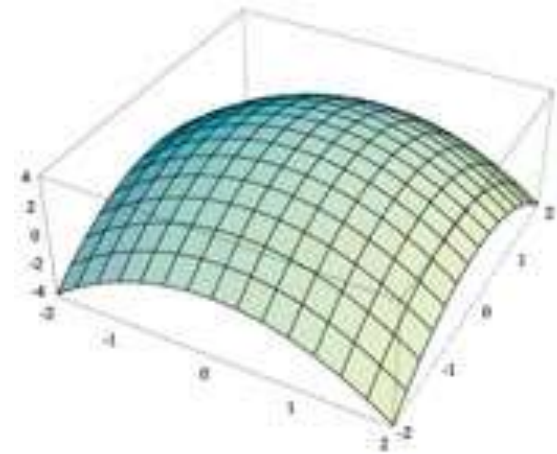
- The main principle of evolution used in GA is “*survival of the fittest*”.
- The good solution survive, while bad ones die.



EXAMPLE PROBLEM I (CONTINUOUS)



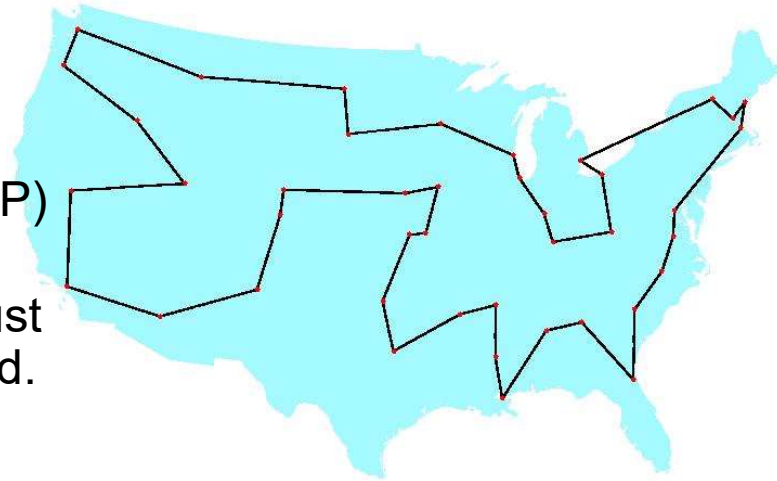
Finding the maximum (minimum)
of some function (within a defined
range).



EXAMPLE PROBLEM II (DISCRETE)

The Traveling Salesman Problem (TSP)
A salesman spends his time visiting n cities. In one tour he visits each city just once, and finishes up where he started. In what order should he visit them to minimize the distance traveled?

There are $(n-1)!/2$ possible tours.

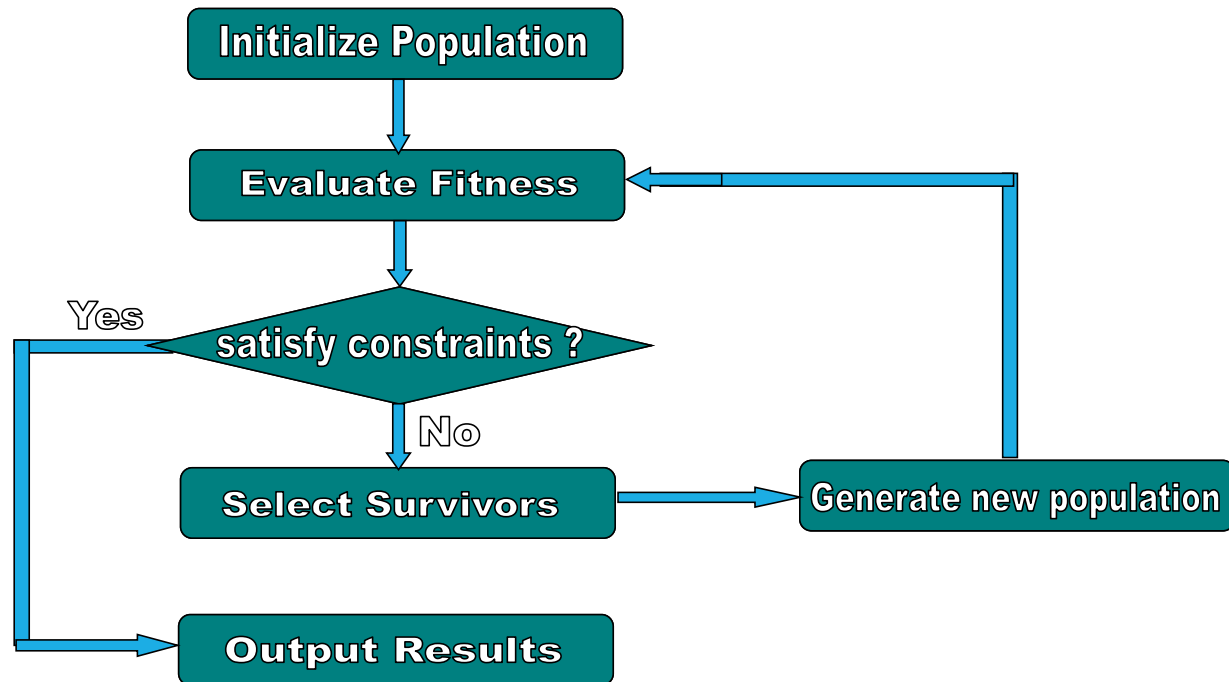


	A	B	C	
A	0	12	34	...
B	12	0	76	...
C	34	76	0	...
...	...	...	...	...

GENETIC ALGORITHM

- Inspired by natural evolution
- Population of individuals
 - *Individual is feasible solution to problem*
- Each individual is characterized by a Fitness function
 - *Higher fitness is better solution*
- Based on their fitness, parents are selected to produce offspring for a new generation
 - *Fitter individuals have more chance to reproduce*
 - *New generation has same size as old generation; old generation dies*
- Offspring has combination of properties of two parents
- If well designed, population will converge to optimal solution

GENETIC ALGORITHM



GENETIC ALGORITHM

Coding or Representation

- Possible solutions to problem

Fitness function

- Parent selection

Reproduction

- Crossover
- Mutation

Convergence

- When to stop

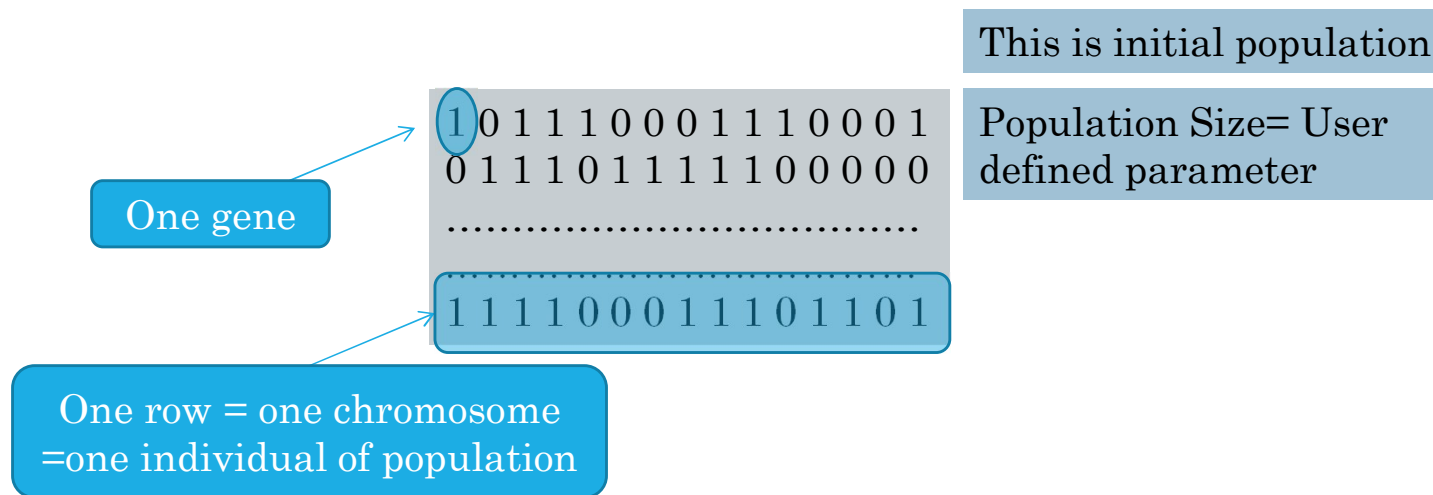
EXAMPLE PROBLEMS

1. Max 1s – 8 bit – find one with highest number of 1s
2. highest decimal value
3. 4- Queens problem
4. Travelling Salesman problem
5. Feature Selection

CODING — EXAMPLE: FEATURE SELECTION

Assume we have 15 features f1 to f15

Generate binary strings of 15 bits as initial population



1 means the feature is used – 0 means the feature is not used

CODING: EXAMPLE

The Traveling Salesman Problem:

Find a tour of a given set of cities so that

- Each city is visited only once
- The total distance traveled is minimized

Representation is an ordered list of city numbers

1) London 3) Dunedin 5) Beijing 7) Tokyo
2) Venice 4) Singapore 6) Phoenix 8) Victoria

CityList1 (3 5 7 2 1 6 4 8)

CityList2 (2 5 7 6 8 1 3 4)

FITNESS FUNCTION/PARENT SELECTION

- Fitness function evaluates *how good* an individual is in solving the problem
- Fitness is computed for each individual
- Fitness function is application depended
- For classification – we may use the classification rate as the fitness function
- Find the fitness value of each individual in the population

FITNESS FUNCTION/PARENT SELECTION

Parent/Survivor Selection

- RouletteWheel Selection
- Tournament Selection
- Rank Selection
- Elitist Selection

ROULETTE WHEEL SELECTION

Main idea: better individuals get higher chance

Individuals are assigned a probability of being selected based on their fitness.

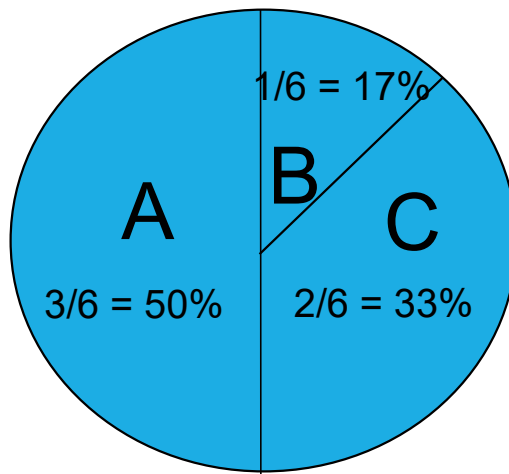
$$p_i = f_i / \sum f_i$$

- Where p_i is the probability that individual i will be selected,
- f_i is the fitness of individual i , and
- $\sum f_i$ represents the sum of all the fitnesses of the individuals with the population.

ROULETTE WHEEL SELECTION

Assign to each individual a part of the roulette wheel

Spin the wheel n times to select n individuals



$\text{fitness}(A) = 3$

$\text{fitness}(B) = 1$

$\text{fitness}(C) = 2$

TOURNAMENT SELECTION

Binary tournament

- Two individuals are randomly chosen; the fitter of the two is selected as a parent

Larger tournaments

- n individuals are randomly chosen; the fittest one is selected as a parent

OTHER METHODS

Rank Selection

- Each individual in the population is assigned a numerical rank based on fitness, and selection is based on this ranking.

Elitism

- Reserve k slots in the next generation for the highest scoring/fittest chromosomes of the current generation

REPRODUCTION

Reproduction operators

- Crossover
- Mutation

Crossover is usually the primary operator with mutation serving only as a mechanism to introduce diversity in the population

REPRODUCTION

Crossover

- Two parents produce two offspring
- There is a chance that the chromosomes of the two parents are copied unmodified as offspring
- There is a chance that the chromosomes of the two parents are randomly recombined (crossover) to form offspring
- Generally the chance of crossover is between 0.6 and 1.0

Mutation

- There is a chance that a gene of a child is changed randomly
- Generally the chance of mutation is low (e.g. 0.001)

CROSSOVER

Generating offspring from two selected parents

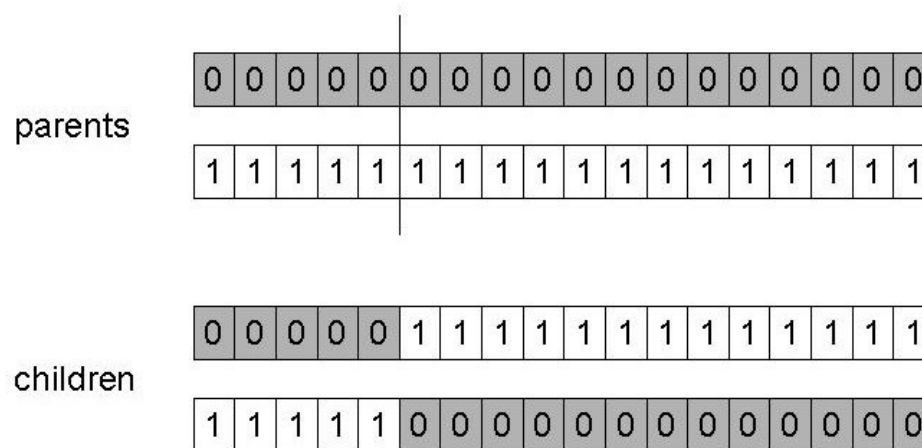
- Single point crossover
- Two point crossover (Multi point crossover)
- Uniform crossover

ONE POINT CROSSOVER

Choose a random point on the two parents

Split parents at this crossover point

Create children by exchanging tails



ONE POINT CROSSOVER

Choose a random point on the two parents

Split parents at this crossover point

Create children by exchanging tails

Parent 1:	X X X X X X X
Parent 2:	Y Y Y Y Y Y Y
Offspring 1:	X X Y Y Y Y Y
Offspring 2:	Y Y X X X X X

CROSSOVER

00101111000110	←
00110010001100	
00101111001000	
11101100000001	←

Crossover point

CROSSOVER

00101111000110	00101100000001
00110010001100	
00101111001000	
11101100000001	11101111000110

CROSSOVER

00101111000110	00101100000001
00110010001100	00110111001000
00101111001000	00101010001100
11101100000001	11101111000110

TWO POINT CROSSOVER

Two-Point crossover is very similar to single-point crossover except that two cut-points are generated instead of one.

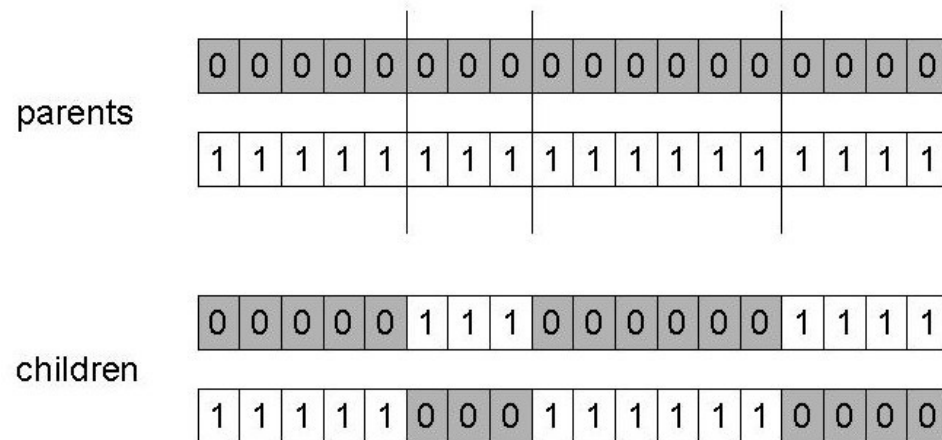
Parent 1:	X X X X X X X
Parent 2:	Y Y Y Y Y Y Y
Offspring 1:	X X Y Y Y X X
Offspring 2:	Y Y X X X Y Y

N POINT CROSSOVER

Choose n random crossover points

Split along those points

Glue parts, alternating between parents



UNIFORM CORSSOVER

- A random mask is generated
- The mask determines which bits are copied from one parent and which from the other parent
- Bit density in mask determines how much material is taken from the other parent

Mask: 0110011000 (Randomly generated)

Parents: 1010001110 0011010010

Offspring: 0011001010 1010010110

MUTATION

Alter each gene independently with a probability p_m

p_m is called the mutation rate

- Typically between $1/\text{pop_size}$ and $1/\text{chromosome_length}$

parent

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

child

0	1	0	0	1	0	1	1	0	0	0	1	0	1	1	0	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

SUMMARY — REPRODUCTION CYCLE

Select parents for producing the next generation

For each consecutive pair apply crossover with probability p_c ,
otherwise copy parents

For each offspring apply mutation (bit-flip with probability p_m)

Replace the population with the resulting population of offsprings

CONVERGENCE

Stop Criterion

- Number of generations
- Fitness value
 - How fit is the fittest individual

GA FOR FEATURE SELECTION

The initial population is randomly generated

Each chromosome is evaluated using the fitness function

The fitness values of the current population are used to find the offsprings of the next generation

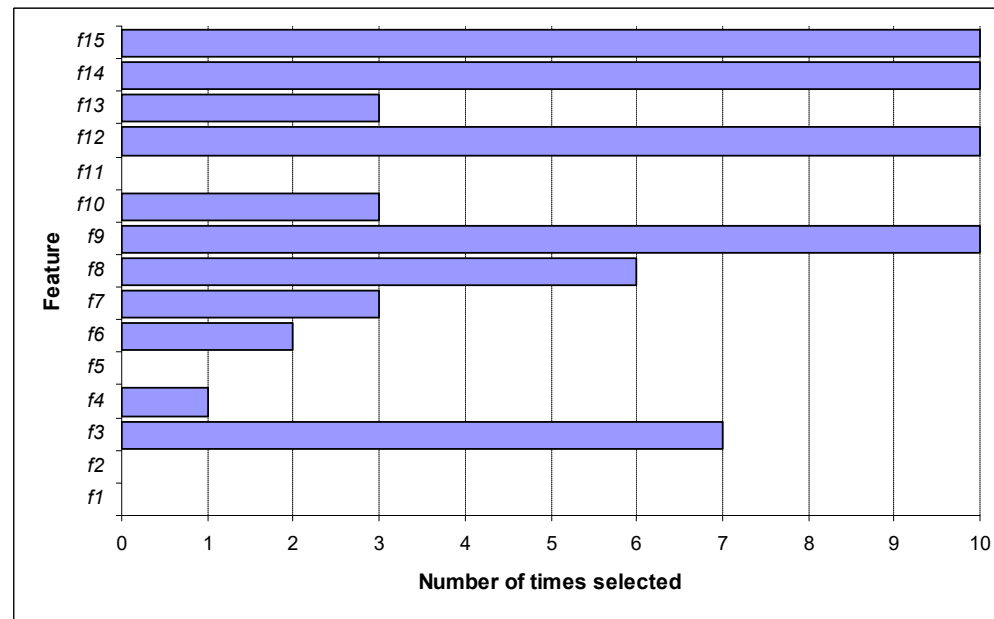
The generational process ends when the termination criterion is satisfied

The selected features correspond to the best individual in the last generation

GA FOR FEATURE SELECTION

GA can be executed multiple times

Example: 15 features, GA executed 10 times



GA FOR FEATURE SELECTION

Feature categories based on frequency of selection

Indispensable:

- Feature selected in each selected feature subset.

Irrelevant:

- Feature not selected in any of the selected subsets.

Partially Relevant:

- Feature selected in some of the subsets.

N QUEEN PROBLEM STATE REPRESENTATION

State / chromosome / individual / survivor / string

Representation also known as encoding i.e., State representation = chromosome encoding

4 3 2 3 → Row position of each queen

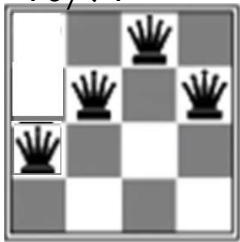


State = "4323" → String of digits, can also be a binary string in which case its called bitstring.



$$h = 2+3+2+3=10$$

$$f=10/91$$



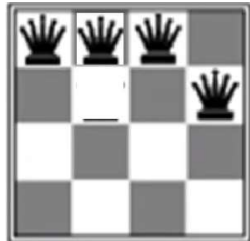
$$h = 2+3+3+2=10$$

$$f=10/91$$



$$h = 0+2+2+2=6$$

$$f=6/91$$



$$h = 2+2+3+1=8$$

$$f=8/91$$



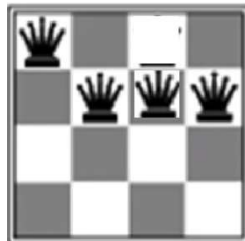
$$h = 1+0+2+1=4$$

$$f=4/91$$



$$h = 1+1+2+1=5$$

$$f = 5/91$$



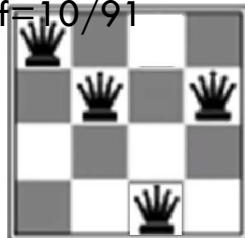
$$h = 1+3+2+2=8$$

$$f=8/91$$



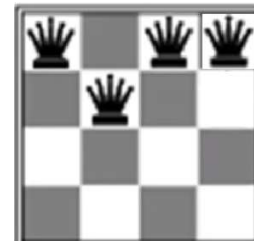
$$h = 2+3+3+2=10$$

$$f=10/91$$



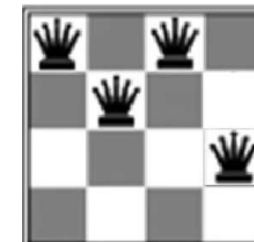
$$h = 1+2+0+1=4$$

$$f=4/91$$



$$h = 3+2+3+2=10$$

$$f=10/91$$



$$h = 2+2+2+0=6$$

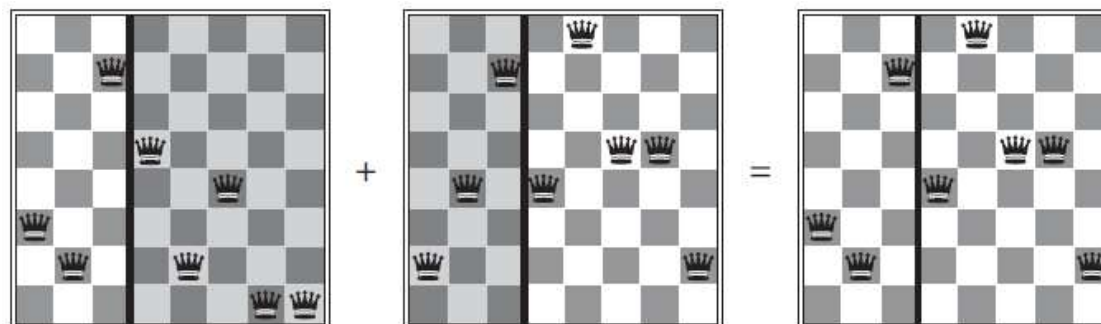
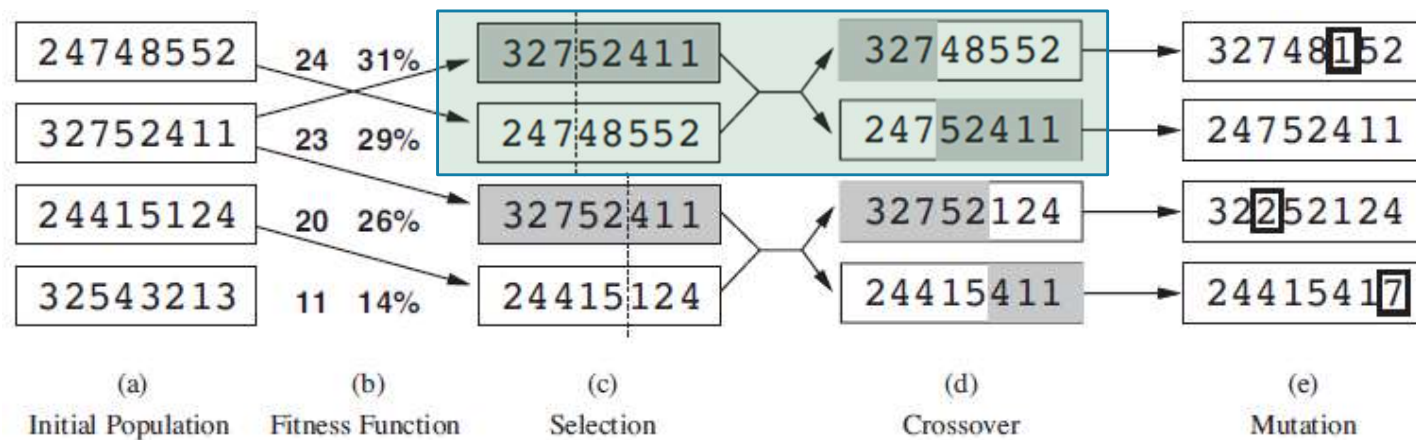
$$f=6/91$$



$$h = 3+3+2+2=10$$

$$f=10/91$$

8 QUEENS PROBLEM: GA SOLUTION



GA WORKED EXAMPLE

- Suppose that we have a rotary system (which could be mechanical - like an internal combustion engine or gas turbine, or electrical - like an induction motor).
- The system has five parameters associated with it - *a, b, c, d and e*. *These parameters can take any integer value between 0 and 10.*
- When we adjust these parameters, the system responds by speeding up or slowing down.
- Our aim is to obtain the highest speed possible in revolutions per minute from the system.

GA WORKED EXAMPLE

Generate a population of random strings (we'll use ten as an example):

<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
1	7	5	2	9
3	8	5	8	4
9	4	8	10	1
2	6	4	8	5
3	6	8	6	9
6	8	5	9	4
7	8	6	4	5
9	10	8	7	5
1	3	5	7	8
9	8	9	6	5

GA WORKED EXAMPLE

- Feed each of these strings into the machine, in turn, and measure the speed in revolutions per minute of the machine. This value is the fitness because the higher the speed, the better the machine:

<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	Fitness
1	7	5	2	9	500
3	8	5	8	4	2000
9	4	8	10	1	100
2	6	4	8	5	2200
3	6	8	6	9	1000
6	8	5	9	4	1100
7	8	6	4	5	750
9	10	8	7	5	330
1	3	5	7	8	800
9	8	9	6	5	1500

GA WORKED EXAMPLE

To select the breeding population, we'll go the easy route and sort the strings then delete the worst ones. First sorting:

<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	Fitness
2	6	4	8	5	2200
3	8	5	8	4	2000
9	8	9	6	5	1500
6	8	5	9	4	1100
3	6	8	6	9	1000
1	3	5	7	8	800
7	8	6	4	5	750
1	7	5	2	9	500
9	10	8	7	5	330
9	4	8	10	1	100

GA WORKED EXAMPLE

Having sorted the strings into order we delete the worst half

<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
2	6	4	8	5
3	8	5	8	4
9	8	9	6	5
6	8	5	9	4
3	6	8	6	9

GA WORKED EXAMPLE

We can now crossover the strings by pairing them up randomly. Since there's an odd number, we'll use the best string twice. The pairs are shown below:

<i>a</i>	<i>b</i>	<i>c</i>	Step 4	
2	6	4	8	5
3	8	5	8	4
9	8	9	6	5
6	8	5	9	4
3	6	8	6	9

2 | 6 4 8 5
 6 | 8 5 9 4 } **Pair 1**

3 6 8 | 6 9
 3 8 5 | 8 4 } **Pair 2**

9 8 | 9 6 5
 2 6 | 4 8 5 } **Pair 3**

GA WORKED EXAMPLE

The crossover points are selected randomly and are shown by the vertical lines. After crossover the strings look like this:

2	8	5	9	4
6	6	4	8	5
3	6	8	8	4
3	8	5	6	9
9	8	4	8	5
2	6	9	6	5

These can now join their parents in the next generation

GA WORKED EXAMPLE

New Generation

<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
2	6	4	8	5
3	8	5	8	4
9	8	9	6	5
6	8	5	9	4
3	6	8	6	9
2	8	5	9	4
6	6	4	8	5
3	6	8	8	4
3	8	5	6	9
9	8	4	8	5
2	6	9	6	5

GA WORKED EXAMPLE

We have one extra string (which we picked up by using an odd number of strings in the mating pool) that we can delete after fitness testing.

<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
2	6	4	8	5
3	8	5	8	4
9	8	9	6	5
6	8	5	9	4
3	6	8	6	9
2	8	5	9	4
6	6	4	8	5
3	6	8	8	4
3	8	5	6	9
9	8	4	8	5
2	6	9	6	5

GA WORKED EXAMPLE

Finally, we have mutation, in which a small number of numbers are changed

<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
2	6	4	8	5
3	8	5	8	4
9	8	9	3	5
6	8	5	9	4
3	6	8	6	9
2	8	5	9	4
6	6	4	8	5
3	6	8	8	4
3	8	5	6	9
9	8	4	8	5
2	6	9	6	5

GA WORKED EXAMPLE

After this, we repeat the algorithm from stage 2, with this new population as the starting point.

Keep repeating until convergence

GA WORKED EXAMPLE

Roulette Wheel Selection

- The alternative (roulette) method of selection would make up a breeding population by giving each of the old strings a chance of ending up in the breeding population which is proportional to its fitness
- Making the fitness for each string the addition of its own fitness with all of those before it

GA WORKED EXAMPLE

Roulette Wheel Selection

<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	Fitness	Cumulative Fitness
2	6	4	8	5	2200	10280
3	8	5	8	4	2000	8080
9	8	9	6	5	1500	6080
6	8	5	9	4	1100	4580
3	6	8	6	9	1000	3480
1	3	5	7	8	800	2480
7	8	6	4	5	750	1680
1	7	5	2	9	500	930
9	10	8	7	5	330	430
9	4	8	10	1	100	100

1. If we now generate a random number between 0 and 10280 we can use this to select strings.
2. If the random number turns out to be between 0 and 100 then we choose the last string.
3. If it's between 8080 and 10280 we choose the first string.
4. If it's between 2480 and 3480 we choose the string 3 6 8 6 9, etc. You don't have to sort the strings into order to use this method.